In [73]:
```
import pandas as pd
dataset1=pd.read_csv("prep.csv",index_col=None)
df2=dataset1
df2 = pd.get_dummies(df2, drop_first=True)

indep_X=df2.drop('classification_yes', axis=1)
dep_Y=df2['classification_yes']
```

In [74]:
```
def split_scalar(indep_X,dep_Y):
        X_train, X_test, y_train, y_test = train_test_split(indep_X, dep_Y, test
        #X_train, X_test, y_train, y_test = train_test_split(indep_X,dep_Y, test

        #Feature Scaling
        #from sklearn.preprocessing import StandardScaler
        sc = StandardScaler()
        X_train = sc.fit_transform(X_train)
        X_test = sc.transform(X_test)

        return X_train, X_test, y_train, y_test
```

In [75]:
```
from sklearn.model_selection import train_test_split
import time
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
import pickle
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
```

In [ ]:

In [94]:
```
def logistic(X_train,y_train,X_test):
        # Fitting K-NN to the Training set
        from sklearn.linear_model import LogisticRegression
        classifier = LogisticRegression(random_state = 0)
        classifier.fit(X_train, y_train)
        classifier,Accuracy,report,X_test,y_test,cm=cm_prediction(classifier,X_t
        return  classifier,Accuracy,report,X_test,y_test,cm
```

In [95]:
```
def svm_linear(X_train,y_train,X_test):

        from sklearn.svm import SVC
        classifier = SVC(kernel = 'linear', random_state = 0)
        classifier.fit(X_train, y_train)
        classifier,Accuracy,report,X_test,y_test,cm=cm_prediction(classifier,X_t
        return  classifier,Accuracy,report,X_test,y_test,cm
```

In [96]:
```
def svm_NL(X_train,y_train,X_test):
```

```
        from sklearn.svm import SVC
        classifier = SVC(kernel = 'rbf', random_state = 0)
        classifier.fit(X_train, y_train)
        classifier,Accuracy,report,X_test,y_test,cm=cm_prediction(classifier,X_t
        return  classifier,Accuracy,report,X_test,y_test,cm
```

In [97]:
```
def Navie(X_train,y_train,X_test):
        # Fitting K-NN to the Training set
        from sklearn.naive_bayes import GaussianNB
        classifier = GaussianNB()
        classifier.fit(X_train, y_train)
        classifier,Accuracy,report,X_test,y_test,cm=cm_prediction(classifier,X_t
        return  classifier,Accuracy,report,X_test,y_test,cm
```

In [98]:
```
def knn(X_train,y_train,X_test):

        # Fitting K-NN to the Training set
        from sklearn.neighbors import KNeighborsClassifier
        classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski',
        classifier.fit(X_train, y_train)
        classifier,Accuracy,report,X_test,y_test,cm=cm_prediction(classifier,X_t
        return  classifier,Accuracy,report,X_test,y_test,cm
```

In [99]:
```
def Decision(X_train,y_train,X_test):

        # Fitting K-NN to the Training set
        from sklearn.tree import DecisionTreeClassifier
        classifier = DecisionTreeClassifier(criterion = 'entropy', random_state
        classifier.fit(X_train, y_train)
        classifier,Accuracy,report,X_test,y_test,cm=cm_prediction(classifier,X_t
        return  classifier,Accuracy,report,X_test,y_test,cm
```

In [100…
```
def random(X_train,y_train,X_test):

        # Fitting K-NN to the Training set
        from sklearn.ensemble import RandomForestClassifier
        classifier = RandomForestClassifier(n_estimators = 10, criterion = 'entr
        classifier.fit(X_train, y_train)
        classifier,Accuracy,report,X_test,y_test,cm=cm_prediction(classifier,X_t
        return  classifier,Accuracy,report,X_test,y_test,cm
```

In [101…
```
def rfe_classification(acclog,accsvml,accsvmnl,accknn,accnav,accdes,accrf):

    rfedataframe=pd.DataFrame(index=['Logistic','SVC','Random','DecisionTree'],c


    for number,idex in enumerate(rfedataframe.index):

        rfedataframe.loc[idex,'Logistic']=acclog[number]
        rfedataframe.loc[idex,'SVMl']=accsvml[number]
        rfedataframe.loc[idex,'SVMnl']=accsvmnl[number]
        rfedataframe.loc[idex,'KNN']=accknn[number]
        rfedataframe.loc[idex,'Navie']=accnav[number]
        rfedataframe.loc[idex,'Decision']=accdes[number]
        rfedataframe.loc[idex,'Random']=accrf[number]
    return rfedataframe
```

In [109…

```
rfelist=rfeFeature(indep_X,dep_Y,2)

acclog=[]
accsvml=[]
accsvmnl=[]
accknn=[]
accnav=[]
accdes=[]
accrf=[]

for i in rfelist:
    X_train, X_test, y_train, y_test=split_scalar(i,dep_Y)


    classifier,Accuracy,report,X_test,y_test,cm=logistic(X_train,y_train,X_test)
    acclog.append(Accuracy)

    classifier,Accuracy,report,X_test,y_test,cm=svm_linear(X_train,y_train,X_tes
    accsvml.append(Accuracy)

    classifier,Accuracy,report,X_test,y_test,cm=svm_NL(X_train,y_train,X_test)
    accsvmnl.append(Accuracy)

    classifier,Accuracy,report,X_test,y_test,cm=knn(X_train,y_train,X_test)
    accknn.append(Accuracy)

    classifier,Accuracy,report,X_test,y_test,cm=Navie(X_train,y_train,X_test)
    accnav.append(Accuracy)

    classifier,Accuracy,report,X_test,y_test,cm=Decision(X_train,y_train,X_test)
    accdes.append(Accuracy)

    classifier,Accuracy,report,X_test,y_test,cm=random(X_train,y_train,X_test)
    accrf.append(Accuracy)

result=rfe_classification(acclog,accsvml,accsvmnl,accknn,accnav,accdes,accrf)
```

```
LogisticRegression()
```

```
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

```
Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
C:\Users\Diwali 6\anaconda3\envs\ml\Lib\site-packages\sklearn\linear_model\_logis
tic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s)
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
```

```
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
SVC(kernel='linear', random_state=0)
RandomForestClassifier(criterion='entropy', n_estimators=10, random_state=0)
DecisionTreeClassifier(max_features='sqrt', random_state=0)
```

In [104…

```
#3
result
```

Out[104…

	Logistic	SVMl	SVMnl	KNN	Navie	Decision	Random
Logistic	0.94	0.94	0.94	0.94	0.94	0.94	0.94
SVC	0.87	0.87	0.87	0.87	0.87	0.87	0.87
Random	0.91	0.92	0.93	0.93	0.86	0.91	0.94
DecisionTree	0.93	0.93	0.94	0.95	0.74	0.95	0.97

In [106…

```
#4
result
```

Out[106…

	Logistic	SVMl	SVMnl	KNN	Navie	Decision	Random
Logistic	0.95	0.95	0.95	0.95	0.95	0.95	0.95
SVC	0.96	0.96	0.96	0.96	0.96	0.96	0.96
Random	0.93	0.93	0.94	0.93	0.91	0.91	0.94
DecisionTree	0.97	0.97	0.97	0.96	0.84	0.96	0.96

In [108…

```
#5
result
```

Out[108…

	Logistic	SVMl	SVMnl	KNN	Navie	Decision	Random
Logistic	0.98	0.98	0.98	0.98	0.98	0.98	0.98
SVC	0.99	0.99	0.99	0.99	0.99	0.99	0.99
Random	0.97	0.97	0.97	0.96	0.87	0.93	0.97
DecisionTree	0.97	0.98	0.98	0.98	0.91	0.96	0.98

In [110…

```
#2
result
```

Out[110…

	Logistic	SVMl	SVMnl	KNN	Navie	Decision	Random
Logistic	0.84	0.84	0.84	0.84	0.84	0.84	0.84
SVC	0.74	0.74	0.74	0.74	0.74	0.74	0.74
Random	0.91	0.92	0.92	0.94	0.85	0.92	0.9
DecisionTree	0.88	0.89	0.9	0.92	0.8	0.89	0.9

In [111…

```
"""
Solution:
The best performance was achieved at k=5 features, where the SVM model reached t

This indicates that retaining more relevant features improves model performance.

Among all models, SVM performed the best, followed by Logistic Regression and De

Naive Bayes showed comparatively lower accuracy due to its assumption of feature

Therefore, SVM with 5 selected features is the optimal model for this dataset.
"""
```

Out[111…

```
'\nSolution:\nThe best performance was achieved at k=5 features, where the SVM
model reached the highest accuracy of 99%.\n\nThis indicates that retaining mor
e relevant features improves model performance.\n\nAmong all models, SVM perfor
med the best, followed by Logistic Regression and Decision Tree.\n\nNaive Bayes
showed comparatively lower accuracy due to its assumption of feature independen
ce.\n\nTherefore, SVM with 5 selected features is the optimal model for this da
taset.\n'
```

In [ ]: